Struktura i dekompozicija grafa

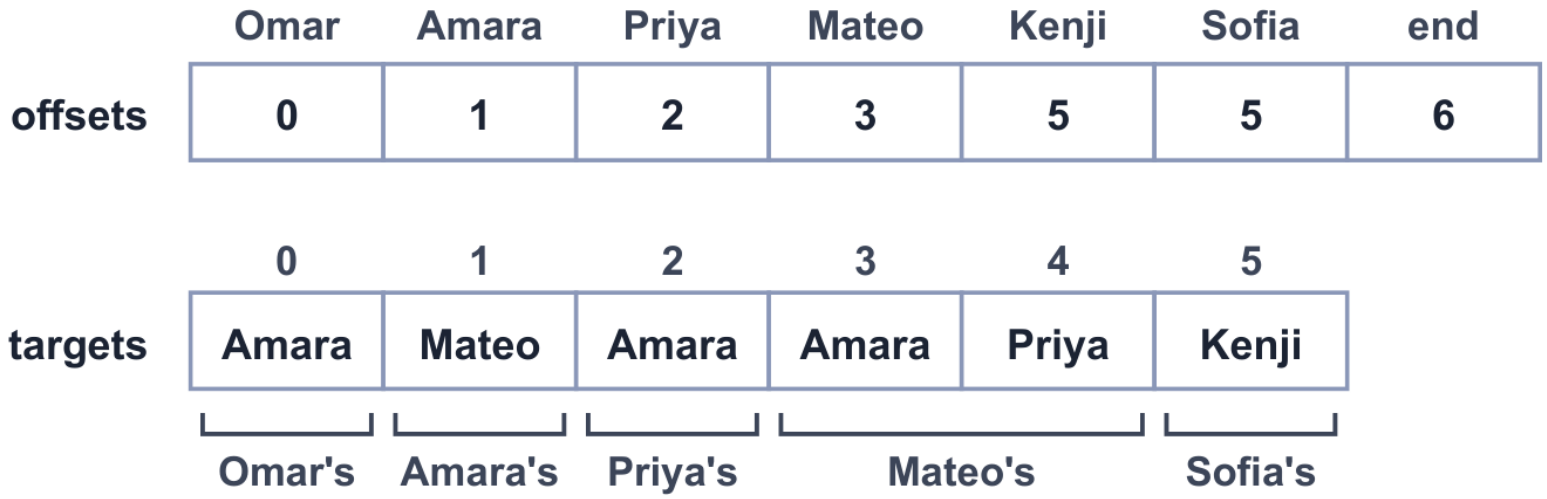
Poredak, čvorovi ovisnosti, slabe točke mreže i spajanje skupova

dr.sc.tech. Kristijan Poje, 2026.

Graf, prikaz i dogovor o smjeru brida

Graf $G = (V, E)$, $n = |V|$ vrhova, $m = |E|$ bridova.

- **Usmjereni graf:** brid (u, v) je jednosmjeran. **U ovom predavanju *u* → *v* znači "*u* prije *v*"** (odnosno: *v* čeka *u*).
- **Neusmjereni graf:** brid $\{u, v\}$ vrijedi u oba smjera (Odjeljak 5 i 6).
- **Put** (path): niz vrhova s bridom između susjednih; **ciklus**: put duljine ≥ 1 koji se vraća u početni vrh bez drugih ponavljanja. *u* **dohvaća** *v* ako postoji put od *u* do *v*; svaki vrh dohvaća sam sebe.
- **Prikaz:** matrica susjedstva ima n^2 ćelija — za milijun vrhova 10^{12} ćelija, gotovo sve prazne. Zato **liste susjedstva**: za svaki vrh redak izlaznih bridova `out[v]` i redak ulaznih bridova `into[v]`. "Kamo mogu iz *v*?" košta deg(*v*) koraka.
- Svi algoritmi danas: **Θ(*n* + *m*)** — jedan pogled na svaki vrh i svaki brid (union-find: skoro konstanta po operaciji).



Karta predavanja: svako pitanje rađa sljedeće

#	algoritam	pitanje	alat	što daje	što ne zna → zato sljedeće
1	Kahn	tko može prvi, kojim redom?	brojanje čekanja + red	poredak ili zaglavljeni skup	koji vrhovi čine ciklus? (Omar nije u njemu)
2	DFS	koji ciklus drži?	šetnja + stanja NOV / NA PUTU / GOTOV	poredak ili jedan imenovani ciklus	svi čvorovi? tko je u čvoru, a tko čeka iza?
3	Algoritam T	tko odlučuje samo zajedno?	DFS + n(v), low(v) + vođa	sve SCC-ove, topološki numerirane	—
4	Kondenzacija, 2-SAT	što s čvorovima?	stegni SCC u vrh → DAG → opet Kahn	poredak čvorova; rješenje 2-SAT-a	samo usmjereni grafovi; ne kaže što puca
5	Blokovi	čiji kvar presijeca mrežu?	ista šetnja, novo značenje low	prijelomne točke, mostovi, blokovi	bridovi stižu uživo?
6	Union-find	jesu li već povezani?	predstavnik + rang + sažimanje puta	povezanost, skoro O(1) po pitanju	koliko dobro povezani? → natrag na blokove

Zajednička nit: ista DFS šetnja (2 → 3 → 5) · isti vođa / predstavnik grupe (3 i 6) · ista kontrakcija: grupa → jedan vrh (4, 5 i 6).

Usmjereni graf: 1–4 ("tko čeka koga"). **Neusmjereni graf:** 5–6 ("tko može kome pisati").

Odjeljak 1 — Topološki poredak: Kahnov algoritam

Pitanje: tko može prvi i kojim redom?

Polazište: graf ovisnosti iz chata.

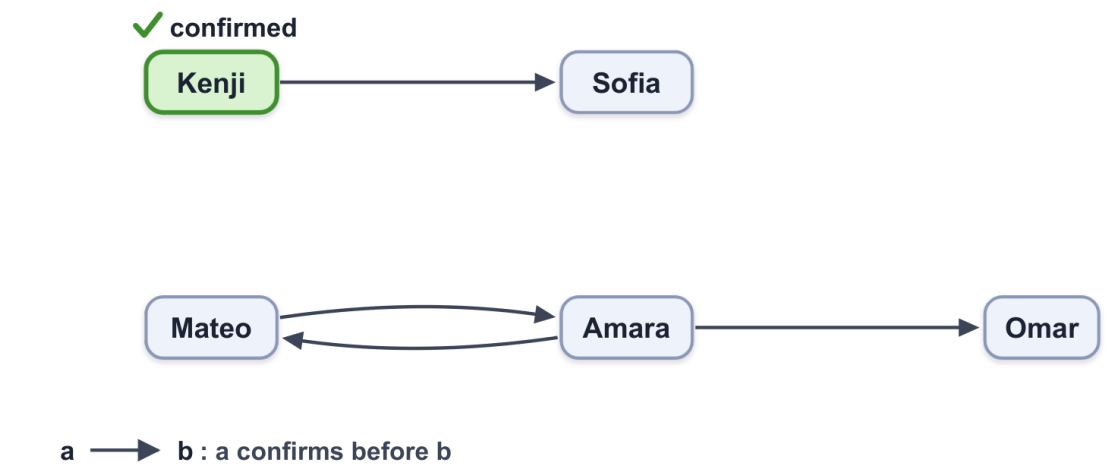
Kahn → DFS → Algoritam T → Kondenzacija / 2-SAT → Blokovi → Union-find

Problem: tko može potvrditi prvi?

Poruke gostiju (ponedjeljak):

- Kenji je potvrdio, bez uvjeta.
- Sofia: "potvrđujem kad Kenji potvrdi" → K → S
- Omar: "kad Amara potvrdi" → A → O
- Amara: "kad Mateo potvrdi" → M → A
- Mateo: "kad Amara potvrdi" → A → M

Definicija: topološki poredak je niz u kojem se svaki vrh pojavljuje točno jednom i **svaki brid pokazuje unaprijed**. Postoji ⇔ graf nema ciklus (DAG). Graf bez ciklusa može imati mnogo poredaka.



Kahnov algoritam: pravilo i invarijante

1. Za svaki vrh izračunaj **broj čekanja** = ulazni stupanj (in-degree) među još nepostavljenim vrhovima.
2. Vrhove s brojem 0 stavi u **red** (FIFO).
3. Dok red nije prazan: uzmi vrh *v* s početka reda, dopiši ga u poredak; svakom *w* iz `out[v]` smanji broj čekanja; ako padne na 0, dodaj *w* na kraj reda.
4. Ako su postavljeni svi vrhovi → **poredak**. Inače → **zaglavljeni skup** (stuck set).

Invarijante: (1) svaki postavljeni vrh dolazi nakon svega što čeka; (2) svaki nepostavljeni vrh broji nepostavljene vrhove koje još čeka; (3) red sadrži točno nepostavljene vrhove s brojem 0.

Tvrdnja: u zaglavljenom skupu svaki vrh još čeka neki vrh iz istog skupa ⇒ šetnja "čeka koga" nikad ne izlazi iz skupa ⇒ skup je konačan, pa se šetnja mora vratiti ⇒ **ciklus postoji**. Zaglavljeni skup je *certifikat* da poretka nema.

Složenost: $\Theta(n + m)$: svaki vrh ulazi i izlazi iz reda najviše jednom, svaki brid smanjuje jedan brojač najviše jednom.

Kahnov algoritam korak po korak

korak	postavljen	O	A	M	K	S	red
start	—	1	1	1	0	1	K
1	K	1	1	1	0	**0**	S
2	S	1	1	1	0	0	(prazan)

Red je prazan, postavljeno 2 od 5 vrhova → **zaglavljeni skup {O, A, M}**: O čeka A, A čeka M, M čeka A — nitko ne izlazi iz skupa.



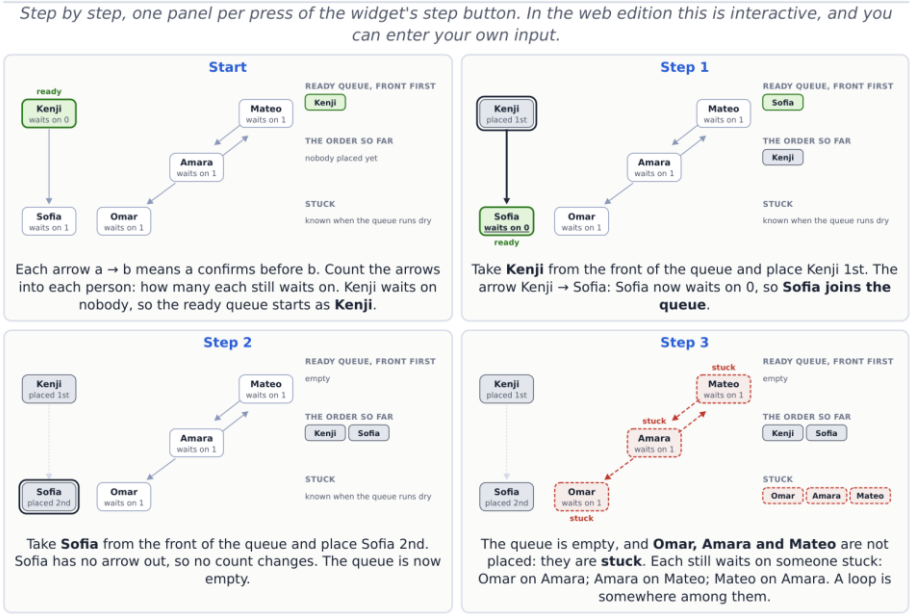
Vježba: bez Mateove poruke

Uklonite Mateovu poruku (brid A → M). Bridovi: K → S, M → A, A → O. Provedite Kahna.

Početni brojevi: O 1, A 1, M 0, K 0, S 1 → red: M, K.

korak	postavljen	oslobođen	red nakon koraka
1	M	A	K, A
2	K	S	A, S
3	A	O	S, O
4	S	—	O
5	O	—	(prazan)

Poredak: **M, K, A, S, O** — svi bridovi unaprijed.



Odjeljak 2 — DFS: poredak ili imenovani ciklus

Pitanje: koji ciklus drži sve?

Zašto: Kahn kaže samo "zaglavljeni {O, A, M}" — ne kaže koji je krug, a Omar nije u njemu.

DFS sa stanjima vrhova

Svaki vrh je u jednom od tri stanja: **NOV** → **NA PUTU** (ušli smo, nismo završili) → **GOTOV**.

- Ulaskom u vrh: stanje NA PUTU, okvir (vrh, indeks sljedećeg brida) na stog.
- Brid $v \to w$: ako je w NOV → spusti se u w. Ako je w NA PUTU → **ciklus**: dio puta od w do v plus brid $v \to w$. Ako je w GOTOV → ništa.
- Kad su svi bridovi vrha obrađeni → vrh je GOTOV, skida se sa stoga.
- Nema ciklusa ⇒ **obrnuti redoslijed završavanja je topološki poredak**.

Složenost: $\Theta(n + m)$; memorija: jedan okvir po vrhu na putu.

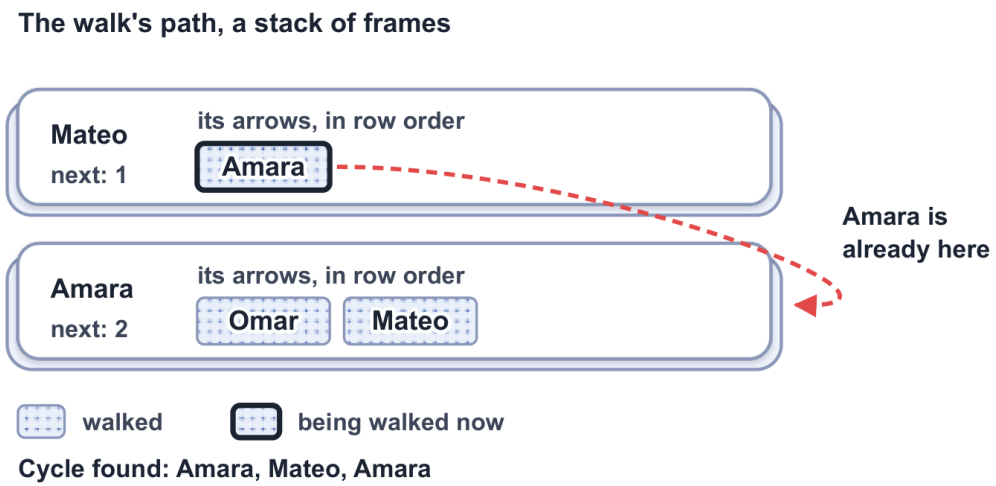
DFS na chatu, korak po korak

Numeracija vrhova: O, A, M, K, S (tim redom kreće šetnja).

korak	događaj	put (stog)
1	uđi u O; O nema bridova → O GOTOV	—
2	uđi u A; brid A → O: O GOTOV, ništa	A
3	brid A → M: M NOV, spusti se	A, M
4	brid M → A: **A je NA PUTU**	A, M

Ciklus: **A → M → A**.

Bez Mateove poruke: završavaju redom O, A, M, S, K → obrnuto **K, S, M, A, O** (drugačiji, ali jednako ispravan poredak od Kahnovog).



Odjeljak 3 — Čvrsto povezane komponente: Algoritam T

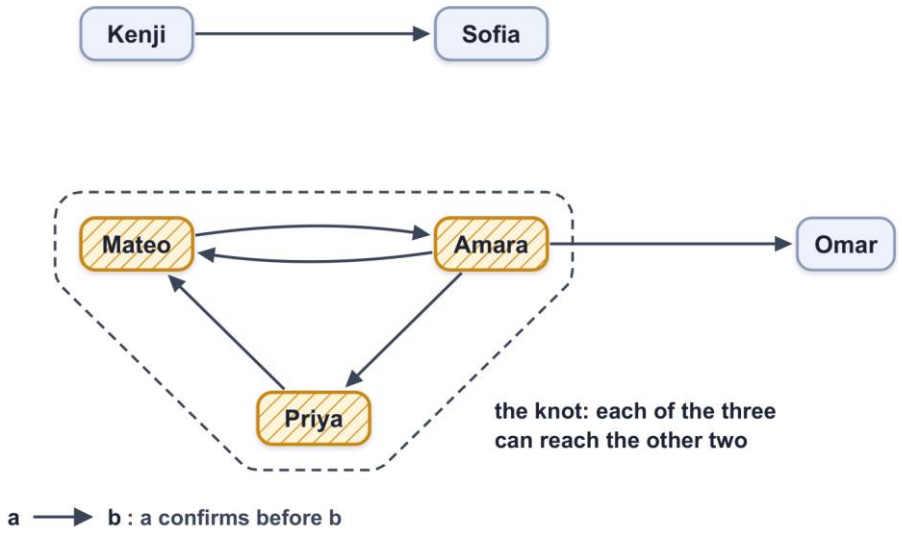
Pitanje: tko odlučuje samo zajedno?

Zašto: DFS imenuje jedan ciklus, ali ne sve čvorove i ne razlikuje člana čvora od onoga koji čeka iza.

Definicija i primjer

Dodane poruke: Priya čeka Amaru (A → P), Mateo čeka i Priyu (P → M). Bridovi sada: K → S, A → O, A → P, A → M, P → M, M → A.

- **Čvrsto povezana komponenta (SCC):** najveći skup vrhova u kojem svaki dohvaća svaki.
- Svaki vrh je u **točno jednoj** komponenti (particija skupa *V*).
- Komponenta s ≥ 2 vrha (ili s petljom) sadrži ciklus — knjiga je zove **čvor** (knot).
- Chat ima 4 komponente: **{A, P, M}** (čvor; ciklus A → P → M → A), {O}, {K}, {S}.
- Kahn bi rekao "zaglavljeni: A, P, M, O" — ali Omar *nije* u čvoru: A dohvaća O, ali O ne dohvaća nikoga.



Zašto SCC, a ne samo detekcija ciklusa?

SCC nije jedan ciklus — to je najveća skupina međusobno slijepljenih ciklusa.

razlog	**detekcija ciklusa**	**SCC (Algoritam T)**
koliko ciklusa?	jedan svjedok, npr. A → M → A — Priya promakne	svi čvorovi u jednom prolazu; inače k popravaka = k prolaza, O(k(n + m))
preklapanje	A → P → M → A i A → M → A — koji je "taj" ciklus?	slijepi ih u jednu jedinicu {A, P, M}
član ili čeka?	Kahn: zaglavljeni {A, P, M, O}	čvor {A, P, M}; O samo čeka iza njega
što dalje?	"nema poretka" → stani	kondenzacija je DAG → poredak čvorova, rad se nastavlja
pitanja o parovima	"postoji li ciklus?" — samo da / ne	"jesu li x i ¬x na zajedničkom ciklusu?" → 2-SAT, O(1) po paru

Detekcija ciklusa: postoji li problem? **SCC:** gdje su svi problemi, tko je unutra i kojim ih redom rješavamo?

Od DFS-a do Algoritma T: ista šetnja + dva broja

	DFS (Odjeljak 2)	**Algoritam T**
pamti po vrhu	stanje NOV / NA PUTU / GOTOV	ulazni broj n(v) i broj povratka low(v)
brid prema vrhu na putu	"ciklus!" → stani	low(v) ← min(low(v), low(w)) → nastavi
brid prema gotovom / izašlom vrhu	ništa	ništa — izašla grupa je zatvorena
završetak vrha	dopiši u redoslijed završavanja	low = n → vođa izlazi s grupom; inače na otvorenu listu
izlaz	poredak ili jedan ciklus	sve SCC-ove, obrnutim topološkim redom
cijena	Θ(n + m)	Θ(n + m) + nekoliko brojeva po vrhu

Ideja: ne staj na prvom ciklusu — zapamti koliko se daleko unatrag svaki vrh može popeti. Tko se ne može popeti iznad sebe, vodi svoju grupu.

Ista šetnja vraća se u Odjeljku 5 (blokovi) — samo low dobiva novo značenje.

Algoritam T: ulazni broj i broj povratka

Jedna DFS šetnja (Tarjan i Zwick, dorada Tarjanova algoritma iz 1972.).

- Pri prvom susretu vrh dobiva **ulazni broj** $n(v) = 1, 2, 3, \ldots$ i **broj povratka** $\text{low}(v) = n(v)$ (low-link; na prikaznicama $p(v)$).
- **Pogled** $v \to w$, w već susretnut i **još nije izašao**: $\text{low}(v) \leftarrow \min(\text{low}(v), \text{low}(w))$.
- **Povratak** iz djeteta w u roditelja v: $\text{low}(v) \leftarrow \min(\text{low}(v), \text{low}(w))$.
- **Završetak** vrha v:
  - $\text{low}(v) = n(v) \to v$ je **vođa**: izlazi zajedno sa svim vrhovima s otvorene liste koji su susretnuti nakon njega — to je cijela komponenta.
  - $\text{low}(v) < n(v) \to v$ ide na **otvorenu listu** (stog) i čeka vođu.
- **Ključno:** brid prema vrhu koji je već izašao **ne mijenja ništa**.

Invarijante: (1) otvorena lista = završeni vrhovi čija komponenta još nije izašla; (2) $\text{low}(v)$ = najraniji ulazni broj do kojeg v može doći unutar svoje komponente; (3) svaka izašla komponenta je cijela.

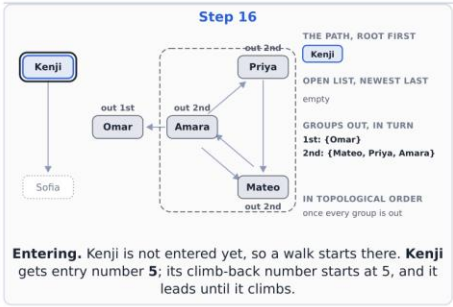
Složenost: $\Theta(n + m)$, nekoliko brojeva po vrhu.

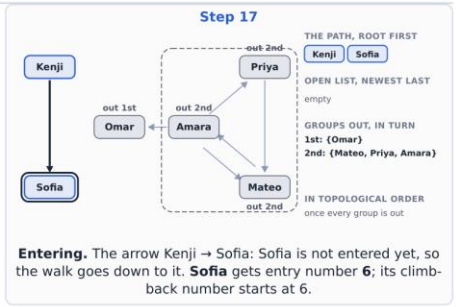
Algoritam T korak po korak

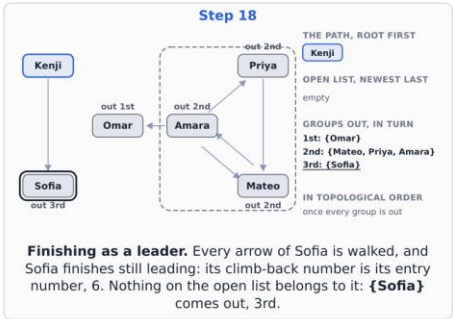
Redoslijed vrhova: O, A, P, M, K, S. Izlazni bridovi od A redom: O, P, M.

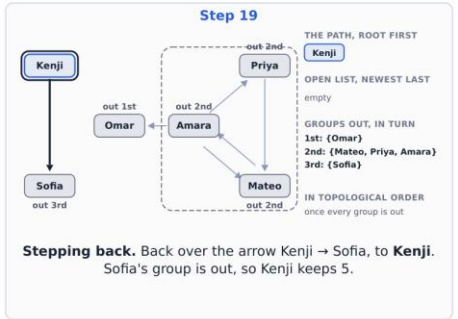
korak	događaj	*n* / low	otvorena lista	izlazi
1	uđi u O; nema bridova; low = *n* → vođa	O: 1/1	—	**1. {O}**
2	uđi u A; A → O: O je izašao → ništa	A: 2/2	—	
3	A → P: uđi u P	P: 3/3	—	
4	P → M: uđi u M	M: 4/4	—	
5	M → A: A susretnut, nije izašao → low(M) = 2	M: 4/**2**	—	
6	M završava: 2 < 4 → na listu		M	
7	povratak u P: low(P) = 2; P završava: 2 < 3 → na listu	P: 3/**2**	M, P	
8	povratak u A: low(A) = min(2, 2) = 2; A → M: M nije izašao, low(M) = 2 nije manji	A: 2/2	M, P	
9	A završava: low = *n* = 2 → vođa; uzima M, P		—	**2. {M, P, A}**
10	uđi u K (5/5); K → S: uđi u S (6/6); S vođa; K vođa		—	**3. {S}, 4. {K}**

Komponente izlaze **obrnutim topološkim redom**; obrnuto numerirano: **K = 0, S = 1, {A, P, M} = 2, O = 3** — svaki brid između komponenti ide od manjeg broja prema većem.











Zašto vođa uzima točno svoju komponentu + vježba

- **Sve što vođa uzme je u njegovoj komponenti:** svaki vrh nakon vođe na listi susretnut je dok je vođa bio na putu, pa ga vođa dohvaća; on nije vođa, pa po invarijanti (2) dohvaća raniji vrh svoje komponente — lanac s padajućim ulaznim brojevima završava u vođi.
- **Ništa iz komponente ne ostaje:** kad bi član komponente bio susretnut prije vođe, neki brid na putu od vođe do njega spustio bi low vođe ispod *n*(vođa) — kontradikcija.
- **Rani prekid:** kad su svi vrhovi susretnuti i low trenutnog vrha padne na low korijena, sve što je ostalo je jedna komponenta.

Vježba: a → b, b → c, c → a, b → d, d → e, e → d (redoslijed susjeda kako je napisan, šetnja od a).

Rješenje: a 1, b 2, c 3; c → a: low(c) = 1; c na listu. Povratak u b: low(b) = 1. b → d: d 4, e 5; e → d: low(e) = 4, e na listu; d završava s low = *n* = 4 → **{e, d}** izlazi. b na listu; a vođa → **{c, b, a}**. Topološki: {a, b, c} → {d, e}.

Odjeljak 4 — Kondenzacija, provjera i 2-SAT

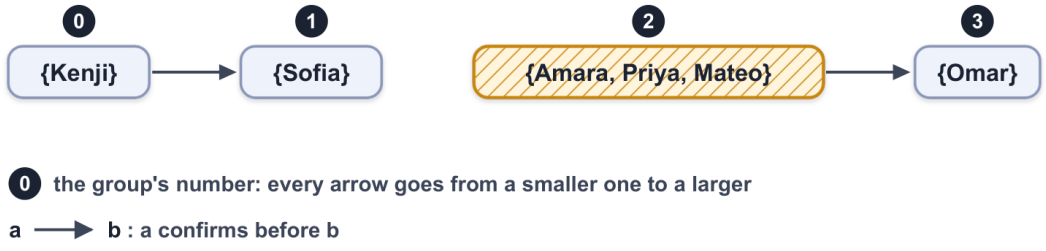
Pitanje: što s čvorovima kad ih imamo?

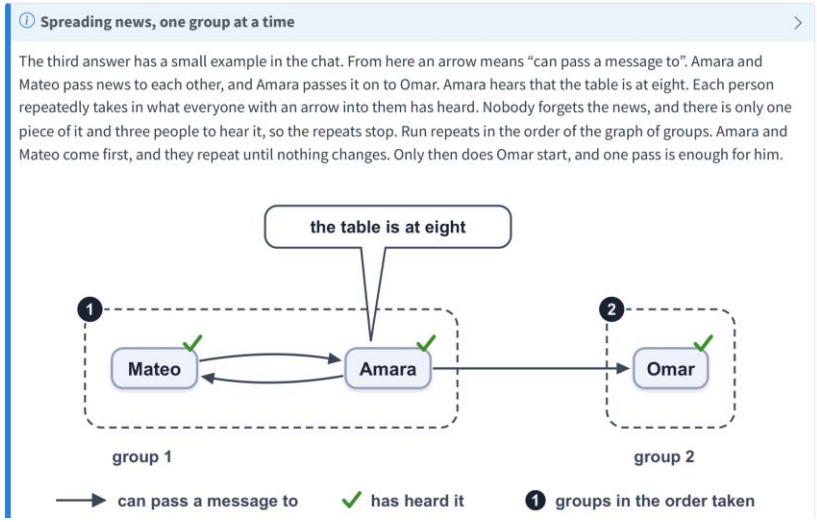
Zašto: stegnemo ih u vrhove i vraćamo se Kahnu; zatim primjena — 2-SAT.

Graf komponenti (kondenzacija)

- Svaku komponentu stegni u jedan vrh; brid $C \to D$ postoji ako neki originalni brid ide iz člana C u člana D ($C \neq D$, svaki brid zadržan jednom).
- **Čuva:** dohvatljivost — *u* dohvaća *v* ⇔ komponenta(*u*) dohvaća komponentu(*v*).
- **Odbacuje:** sve unutar komponente (tko koga čeka između A, P, M).
- **Tvrdnja:** kondenzacija **nema ciklusa**. (Da ga ima, sve komponente na njemu dohvaćale bi jedna drugu → bile bi jedna komponenta.) Zato je uvijek DAG i Algoritam T ju je već numerirao topološki.
- Chat: {K} → {S}, {A, P, M} → {O}.

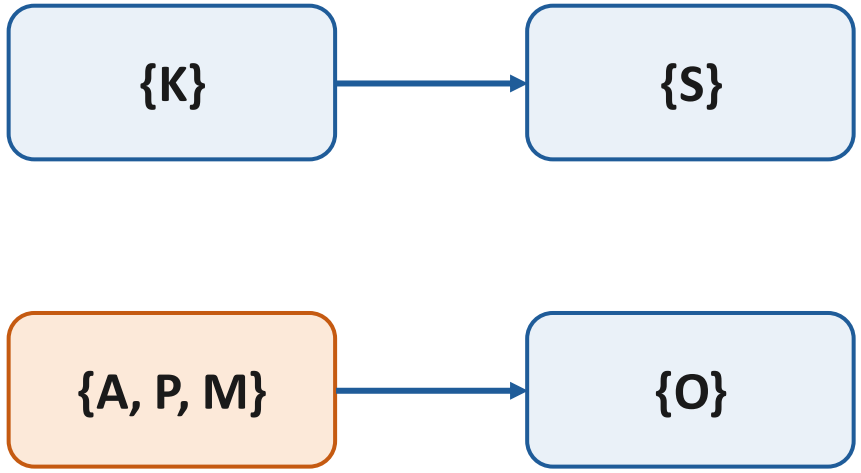
Tri pravila za čvor (ovisno o značenju brida): (1) **odbij** — build sustav: čvor se nikad ne može izgraditi, javi pogrešku koja ga imenuje; (2) **obradi kao jednu jedinicu** — Lean prevodi međusobno rekurzivne funkcije zajedno, LLVM optimizira komponentu po komponentu; (3) **ponavljaj unutar čvora dok se ništa ne mijenja** — monotona fiksna točka (širenje vijesti).





Natrag na Kahna: kondenzacija uvijek ima poredak

Graf komponenti srijedinog chata



Algoritam T: K = 0, S = 1, {A, P, M} = 2, O = 3

Kahn: {K}, {A, P, M}, {S}, {O}

Oba poretka su ispravna.

- **Odjeljak 1:** Kahn na originalnom grafu zapne — zaglavljeni {A, P, M, O}, poretka nema.
- **Sada:** svaki čvor stegnut u jedan vrh. Kondenzacija nema ciklusa, pa Kahn postavi sve.
- **Algoritam T** poredak daje besplatno: obrnuti redoslijed izlaska grupa.
- **Krug se zatvara:** imamo poredak čvorova umjesto poretka vrhova. Unutar čvora odlučuje pravilo: odbij · jedna jedinica · ponavljaj do fiksne točke.

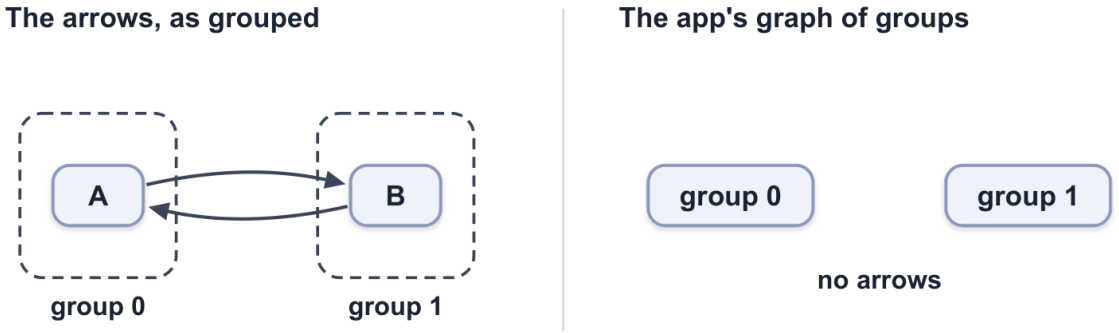
Kako provjeriti tuđe grupiranje

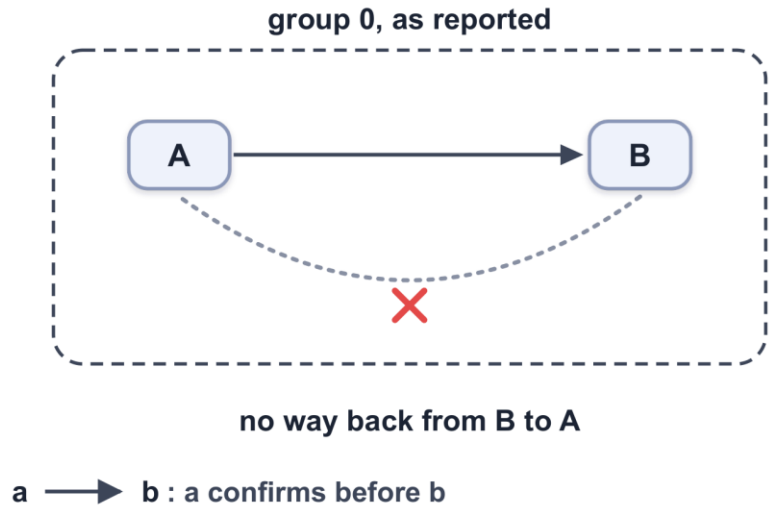
Aplikacija vrati grupiranje. Bez povjerenja u njezin kod dovoljne su **tri provjere**:

1. **Particija:** svaki vrh ima točno jednu grupu, nijedna grupa nije prazna.
2. **Čvrsta povezanost:** iz jednog člana pretraga *po* bridovima, unutar grupe, dohvaća sve članove — i pretraga *protiv* bridova (retci `into`) također.
3. **Aciklička kondenzacija: sami** izgradite graf grupa iz originalnih bridova i pustite Kahna.

Podmetnuti krivi odgovori: (a) A ⇄ B prijavljeni kao dvije grupe + aplikacijin graf grupa bez bridova → prolaze 1 i 2; **pada 3**, ali samo ako graf grupa gradite sami. (b) A → B prijavljeni kao jedna grupa → **pada samo 2** (pretraga protiv bridova iz A ne nalazi B).

Složenost provjere: isti red veličine kao izračun, $\Theta(n + m)$.

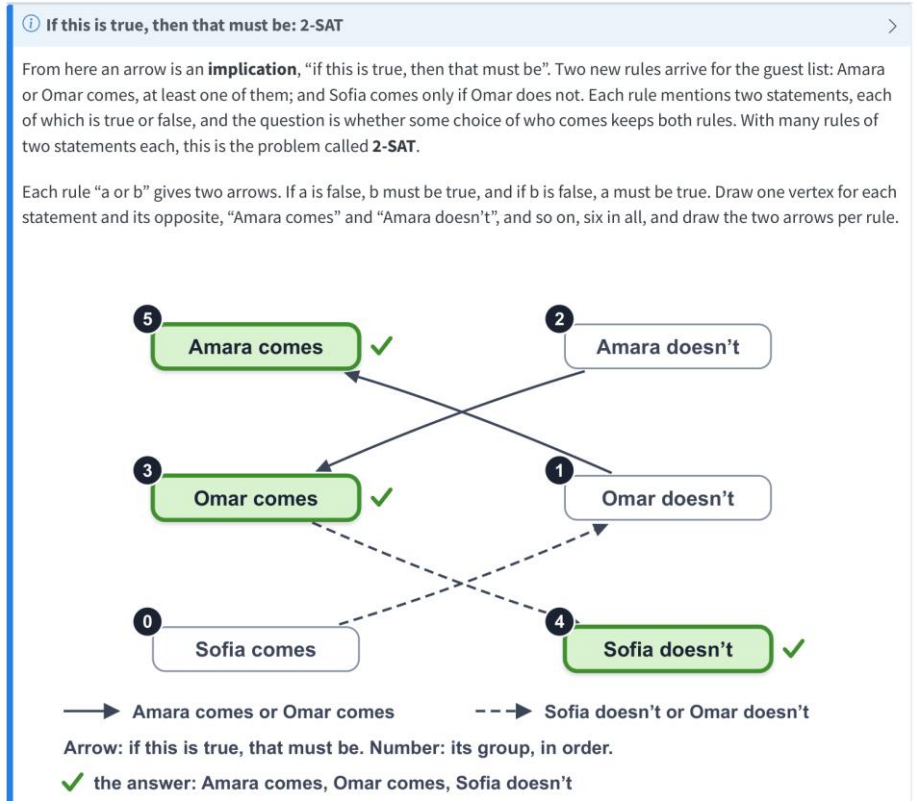




Primjena: 2-SAT kroz komponente

Pravila: (A ∨ O) — dolazi barem jedan od Amare i Omara; (¬S ∨ ¬O) — Sofia dolazi samo ako Omar ne dolazi.

- Svaka klauzula ($a \vee b$) daje dva **implikacijska** brida: $\neg a \to b$ i $\neg b \to a$.
- Vrhovi: A, ¬A, O, ¬O, S, ¬S (6). Bridovi: ¬A → O, ¬O → A, S → ¬O, O → ¬S.
- **Tvrdnja:** nezadovoljivo ⇔ neki x i $\neg x$ u istoj komponenti.
- Inače: x = istina ako komponenta od x dolazi **kasnije** u topološkom poretku od komponente $\neg x$.
- Ovdje: **A dolazi, O dolazi, S ne dolazi.** Složenost $\Theta(n + m)$.



Odjeljak 5 — Mostovi, prijelomne točke i blokovi

Pitanje: čiji pojedinačni kvar presijeca mrežu?

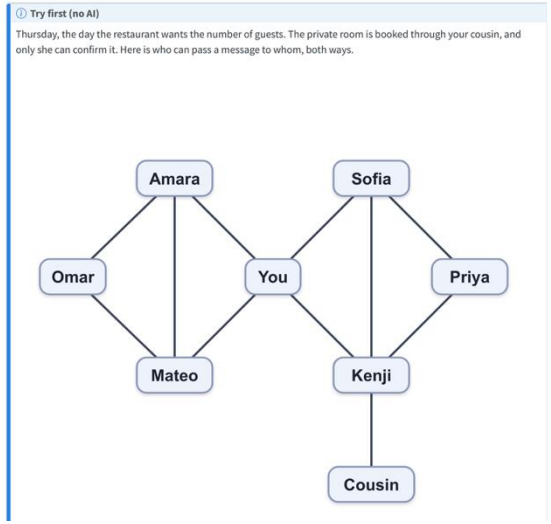
Zašto: dosad je sve bilo usmjereno; sada neusmjerena mreža — ista DFS šetnja, novo značenje low.

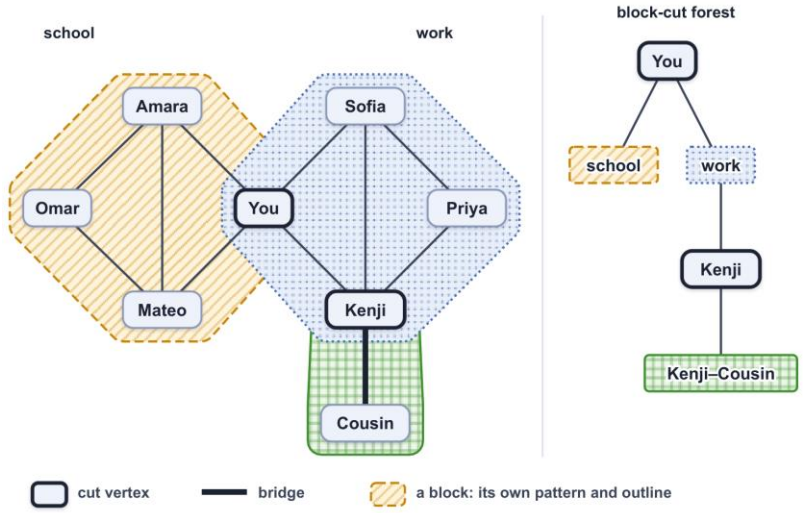
Definicije

Graf je sada **neusmjeren** ("može poslati poruku"). Bridovi: Ti–A, A–M, M–Ti, M–O, O–A, Ti–S, S–K, K–Ti, K–P, P–S, K–R (R = rođakinja).

- **Prijelomna točka** (articulation point, cut vertex): vrh čijim uklanjanjem graf ima više komponenti povezanosti nego prije.
- **Most** (bridge): brid čijim uklanjanjem graf ima više komponenti nego prije.
- **Blok** (biconnected component): najveći podgraf koji nijedan pojedinačni kvar *unutar* njega ne razdvaja. U bloku s ≥ 3 vrha svaka dva vrha spajaju dva puta bez zajedničkih unutarnjih vrhova. Most sa svojim krajevima je blok.
- Dva bloka dijele najviše jedan vrh, i taj je vrh prijelomna točka.
- **Blok-rez šuma:** čvor za svaki blok, čvor za svaku prijelomnu točku, brid od prijelomne točke do svakog njezinog bloka.

Odgovor: prijelomne točke **Ti** i **K**, most **K–R**, blokovi {Ti, A, M, O}, {Ti, S, K, P}, {K, R}.





Šetnja za blokove: test roditelja i pravilo korijena

Ista DFS šetnja s $n(v)$ i low(v), ali novo značenje:

- low(v) = najraniji ulazni broj do kojeg dio stabla ispod v (uključujući v) može doći koristeći **najviše jedan brid izvan stabla**, a **nikad brid kojim je stigao** (preskače se po ID-u brida, ne po vrhu!).
- Brid prema novom vrhu = brid stabla → spusti se. Brid prema ranijem vrhu = povratni brid → low(v) ← min(low(v), $n(w)$).
- Svaki istraženi brid ide na **stog bridova**.
- **Test roditelja** (kad dijete w završi pod u): low(u) ← min(low(u), low(w)), zatim
  - low(w) ≥ $n(u)$ → skini sa stoga bridove do (u, w): to je **jedan blok**; ako u nije korijen → u je **prijelomna točka**;
  - low(w) > $n(u)$ → (u, w) je **most**.
- **Pravilo korijena:** korijen je prijelomna točka ⇔ ima ≥ 2 djeteta u DFS stablu.

Složenost: $\Theta(n + m)$; memorija $\Theta(n + m)$ (stog bridova).

Šetnja na mreži, korak po korak

Korijen Ti. Ulazni brojevi: Ti 1, A 2, M 3, O 4, S 5, K 6, P 7, R 8.

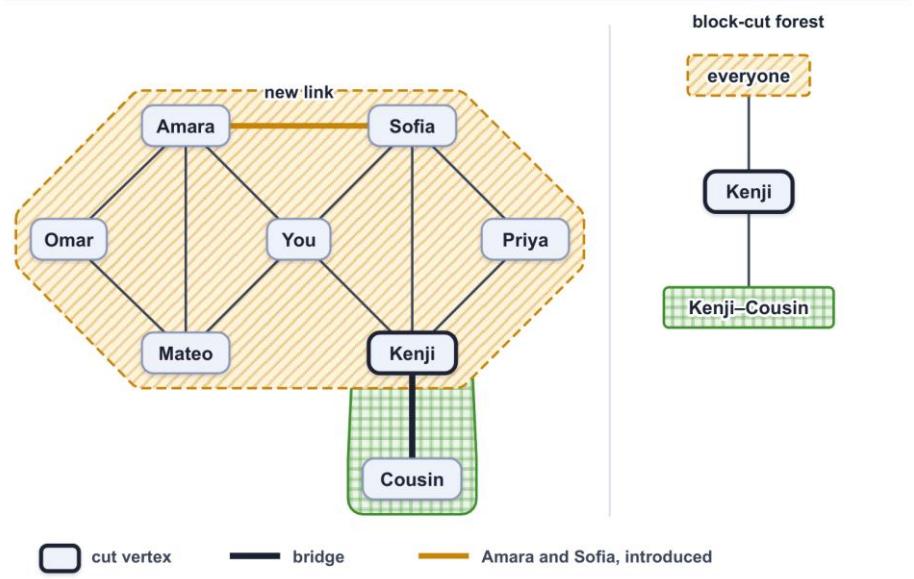
korak	događaj	low	odluka
1	Ti → A → M (bridovi stabla)		
2	M–Ti: povratni brid	low(M) = 1	
3	M → O; O–A: povratni brid	low(O) = 2	
4	O završava pod M	$2 < n(\mathrm{M}) = 3$	nema bloka; low(M) = 1
5	M završava pod A	$1 < n(\mathrm{A}) = 2$	nema bloka; low(A) = 1
6	A završava pod Ti	$1 \geq n(\mathrm{Ti}) = 1$	**blok {Ti, A, M, O}**
7	Ti → S → K; K–Ti: povratni	low(K) = 1	
8	K → P; P–S: povratni; P završava pod K	low(P) = 5 < 6	nema bloka
9	K → R; R završava pod K	low(R) = 8 ≥ 6 i 8 > 6	**blok {K, R}**, K prijelomna, **K–R most**
10	K završava pod S	1 < 5	nema bloka; low(S) = 1
11	S završava pod Ti	1 ≥ 1	**blok {Ti, S, K, P}**
12	Ti je korijen s 2 djeteta (A, S)		**Ti prijelomna**

Vježba i ograničenje

Vježba: u petak se upoznaju Amara i Sofia (novi brid A–S). Koje su sada prijelomne točke i mostovi?

Rješenje: Školski i poslovni prijatelji sada se dodiruju dvaput (kroz Ti i kroz A–S) → blokovi {Ti, A, M, O, S, K, P} i {K, R}; **Ti više nije prijelomna točka**, K jest, K–R je i dalje most.

- **Ograničenje (Assumes):** algoritam odgovara za **jedan** kvar istovremeno. Mreža bez mosta i bez prijelomne točke može se raspasti kad otkažu dva elementa odjednom.
- Primjene: IETF standard za brzo preusmjeravanje (rezervni put za kvar jednog kabela/routera); Hopcroft–Tarjan test planarnosti testira blok po blok.
- Mjereno: 10 000 vrhova, 20 000 bridova — šetnja 720 µs, "obriši svaki brid i prebroji komponente" 1,81 s.



Odjeljak 6 — Union-find

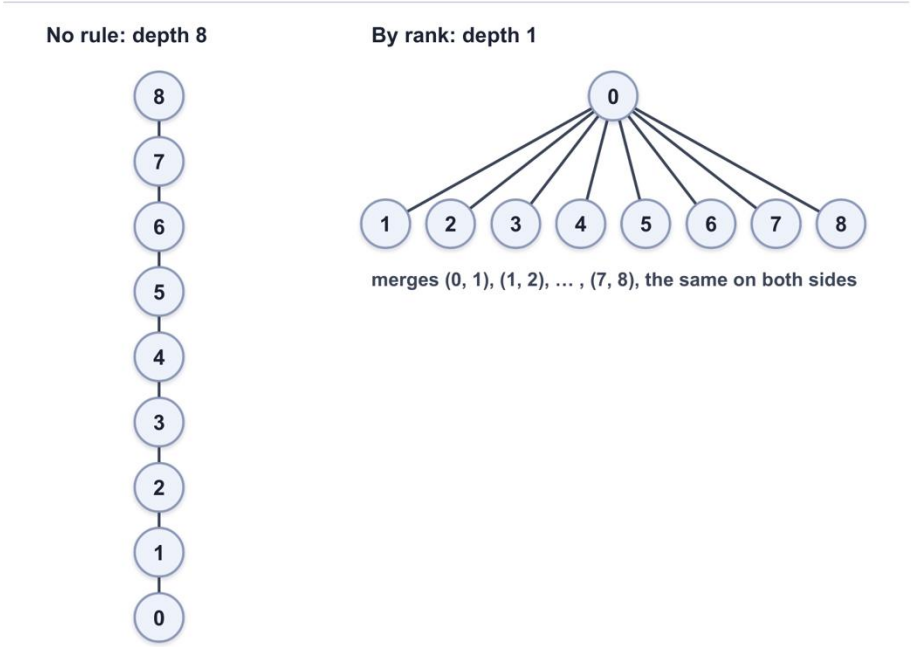
Pitanje: jesu li x i y već povezani — dok bridovi stižu?

Zašto: šetnja radi na gotovom grafu; ponavljati je nakon svakog novog brida preskupo.

Problem i operacije

Bridovi stižu jedan po jedan cijelu večer; nakon svakog netko pita: **jesu li *x* i *y* već povezani?** DFS nakon svakog brida: O(*n* + *m*) po pitanju — preskupo.

- **Šuma nepovezanih skupova:** svaka grupa je stablo; svaki element pokazuje na roditelja; **korijen** pokazuje sam na sebe i predstavlja grupu.
- `make_set(x)`: *x* je sam svoj roditelj, rang 0.
- `find(x)`: slijedi roditelje do korijena.
- `union(x, y)`: nađi oba korijena; ako su isti → ništa, odgovor "ne"; inače jedan korijen objesi ispod drugoga, odgovor "da".
- **Problem:** bez pravila se gradi lanac. Spajanja (0,1), (1,2), …, (7,8) uvijek ispod pridošlice → dubina 8; za *n* elemenata jedan `find` košta *n* − 1.



Dva pravila: unija po rangu i sažimanje puta

1. **Unija po rangu:** korijen manjeg ranga ide ispod korijena većeg ranga; rangovi se ne mijenjaju. Kod jednakih rangova drugi ide ispod prvoga, a rang prvoga raste za 1.
2. **Ista grupa se ne spaja sama sa sobom:** isti korijeni → ništa.
3. **Sažimanje puta (path compression):** na povratku iz korijena svaki element kojim je `find` prošao pokazuje izravno na korijen.

Invarijante: (1) slijeđenje roditelja iz bilo kojeg elementa završava u korijenu njegove grupe; (2) rangovi rastu prema korijenu, a korijen ranga r ima barem 2^r elemenata.

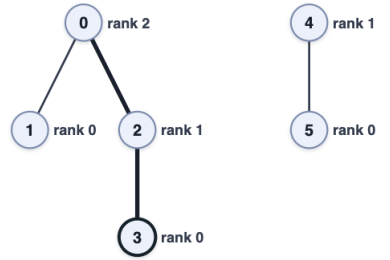
Tvrdnje:

- Dubina stabla $\leq \log_2 n \Rightarrow$ jedan `find` $\leq \log_2 n$ koraka (za milijardu elemenata < 30).
- Niz od m operacija nad n elemenata: $\mathbf{O(m\,\alpha(n))}$, gdje je α inverz Ackermannove funkcije — $\alpha(n) \leq 4$ za svaki n koji stane u računalo. To je **amortizirana** cijena.

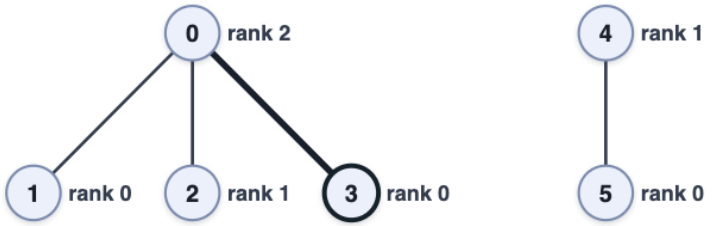
Union-find na šest gostiju, korak po korak

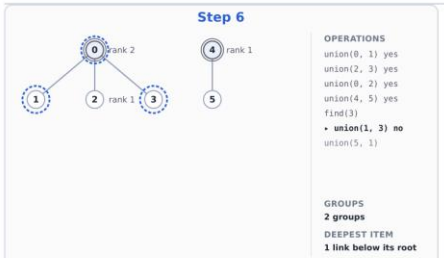
korak	operacija	roditelji 0–5	rangovi 0–5	grupa
start	—	0, 1, 2, 3, 4, 5	0, 0, 0, 0, 0, 0	6
1	union(0, 1): jednaki rangovi, 1 ispod 0	0, 0, 2, 3, 4, 5	1, 0, 0, 0, 0, 0	5
2	union(2, 3): jednaki, 3 ispod 2	0, 0, 2, 2, 4, 5	1, 0, 1, 0, 0, 0	4
3	union(0, 2): jednaki, 2 ispod 0	0, 0, 0, 2, 4, 5	2, 0, 1, 0, 0, 0	3
4	union(4, 5): jednaki, 5 ispod 4	0, 0, 0, 2, 4, 4	2, 0, 1, 0, 1, 0	2
5	find(3): 3 → 2 → 0, zatim 3 pokazuje na 0	0, 0, 0, **0**, 4, 4	2, 0, 1, 0, 1, 0	2
6	union(1, 3): oba korijena 0 → "ne"	bez promjene	bez promjene	2
7	union(5, 1): rang 1 protiv 2, 4 ispod 0	0, 0, 0, 0, **0**, 4	2, 0, 1, 0, 1, 0	1

Gosti iz Odjeljka 5: 0 = Mateo, 1 = Ti, 2 = Omar, 3 = Amara, 4 = Sofia, 5 = Kenji — svaki union je brid mreže.

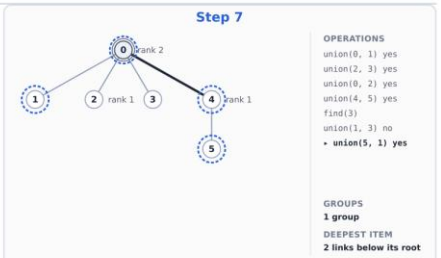


find(3) walks up from 3 through 2 to 0





union(1, 3): looking up 1, 1 → 0 ends at root 0; looking up 3, 3 → 0 ends at root 0. Both end at root 0: **already together**, nothing merges, and the answer is no. Still 2 groups.

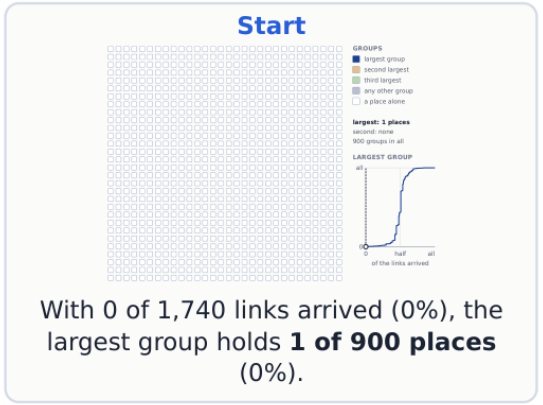


union(5, 1): looking up 5, 5 → 4 ends at root 4; looking up 1, 1 → 0 ends at root 0. Root 4 has the lower rank (1 against 2), so **4 goes under 0**; no rank changes. 2 groups become 1.

Što union-find ne zna

- **Povezano ≠ teško razdvojivo:** nakon A–S union-find kaže "A i rođakinja su povezani" — kao i prije. Ne vidi da je Ti prestao biti prijelomna točka (vježba A–S, Odjeljak 5). Za to treba šetnja za blokove.
- **Odbacuje put:** zna *da* su povezani, ne *kroz koga*.
- **Samo spajanje:** ako se Kenji i Sofia posvađaju i obrišu brojeve, union-find i dalje kaže "povezani" → za brisanje bridova treba *dinamička povezanost* (frontier map, 5.12.4). Poništavanje *zadnjeg* spajanja moguće je bez sažimanja puta (dodatak 5.14).
- Primjene: Kruskalov MST (poglavlje o putevima), zaključivanje tipova u Rustovom prevoditelju, simulacija perkolacije.
- Mjereno: 20 000 upoznavanja s pitanjem nakon svakog — union-find 254 µs, pretraga nakon svakog 639 ms.

Step by step, one panel per press of “Next step”. In the web edition this is interactive, and you can enter your own keys.



Odjeljak 7 — Sažetak

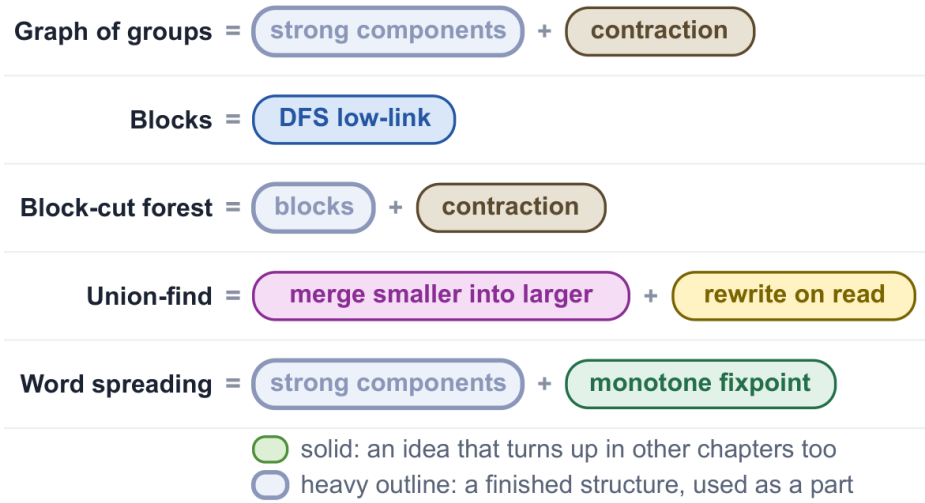
Pet pitanja, pet slika grafa

Svaka slika čuva točno ono što pitanje treba, a ostalo odbacuje.

Kahn → DFS → Algoritam T → Kondenzacija / 2-SAT → Blokovi → Union-find

Pet pitanja, pet slika — i šest zajedničkih ideja

pitanje	slika (struktura)	što odbacuje	vrijeme	ako pretpostavka padne
tko može prvi, kojim redom?	topološki poredak ili zaglavljeni skup (Kahn)	koji od mnogih poredaka; koji ciklus	$\Theta(n + m)$	brid dodan tijekom rada → vrh postavljen prije onoga koga sada čeka
koji ciklus drži?	jedan imenovani ciklus (DFS)	sve ostale cikluse	$\Theta(n + m)$	brid dodan tijekom rada može pokazivati unatrag
tko odlučuje samo zajedno?	SCC + kondenzacija (Algoritam T)	odnose unutar komponente	$\Theta(n + m)$	dvije komponente koje se sada dohvaćaju ostanu razdvojene
čiji pojedinačni kvar presijeca mrežu?	mostovi, prijelomne točke, blok-rez šuma	učinak dvaju kvarova	$\Theta(n + m)$	dva kvara odjednom razdvoje "sigurnu" mrežu
jesu li već povezani?	union-find	put; broj puteva	$O(\alpha(n))$ amortizirano	nakon brisanja brida i dalje kaže "povezani"



Zajedničke ideje: **topološki poredak** · **DFS ulazni broj + low-link** · **kontrakcija** · **manje ispod većega** · **prepiši pri čitanju** · **monotona fiksna točka**.